SKETCHPAD

The Remaining Phases

From the rest of Phase 8 to the end of the list: what each phase will do

Built 27 October 2026 from claude/docs at 80fe622.
The authority is `PHASES.md`.

Contents

CONTENTS

How to read this 3

The list at a glance 4

Phase 8, next pass: drop a handle on a handle to weld the rest of Phase 8 5

Phase 8b: magic points more places a disc can go 7

Phase 9: print aids scale bar, nail spacing, printing at size 9

Phase 10: save and reopen a figure Save As, category, manifest, reload; undo 11

Phase 11: figure follow-ups a collection; you choose what goes in 13

Phase 12: thumbnails in the Insert list icons, larger preview on hover 15

Phase 13: sweep tool no description on record 16

Phase 14: filesystem boundary refactor (C3) one place where the app talks to files 17

Phase 15: older tabs on the figuresCommands pattern new today: investigate first 18

The end-of-work list nine items, help pages last 19

How to read this

← Contents

This covers every phase still to do, starting with the rest of Phase 8. The list and its order are yours (`PHASES.md`); this document only says, for each phase, what would actually be built, which files it touches, what it depends on, how it would be tested, and what has to be decided before it starts.

Two kinds of text are mixed here and kept apart on purpose:

- **Recorded.** What you decided or said, as it stands in `PHASES.md` and the planning notes.
- **Proposed.** My plan for how to do it. Marked "Proposed" where it matters. Nothing proposed here is decided.

Each phase opens with a grey box: its status, what it needs first, its rough size (small: a day's session; medium: two or three; large: more, or several passes), and the files it is expected to touch.

The working rules are unchanged: one phase at a time, each on its own branch, design discussion before building, tests (`npm run test:tabs`) extended every phase, and a stop for your review and "go" after each.

One gap in the record. `PHASES.md` says your "Deferred" and "Dropped" lists belong in it but were never pasted in. I could not find them anywhere I can see. Phase 13 (the sweep tool) has no description anywhere either; see that section.

The list at a glance

← Contents

Phase	What	Status
8, next pass	A handle dropped on another handle welds; the weld + made visible; the centroid ignores the right button	Specified; six questions open
8b	Magic points: midpoints, tops of loops, cusps (and perhaps crossings) as places a disc can go	Not started; needs design
9	Print aids: scale bar, minimum nail-spacing check, printing at size	Not started; needs your answers
10	Save and reopen a figure (Save As, category, manifest); undo proposed alongside	Not started; part of the Node side exists
11	Figure follow-ups: delete a layer, the grayed menu items, more Break apart, the curved objects	A collection; you choose what goes in
12	Thumbnails in the Insert list, larger preview on hover	Not started
13	Sweep tool	No description on record
14	Filesystem boundary refactor (C3)	Not started
15	Investigate redoing the older tabs on the `figuresCommands.js` pattern	New today; investigate first
End list	Nine end-of-work items, ending with the HTML help pages	Not started

Dependencies worth knowing. Phase 9 is easier after 10 (you print a saved figure, not a scratch one). Undo shares its hardest piece with 10. Phase 14 would change how 10 talks to Node, so 10 should be written against a small, clean call (one `saveFigure`), which 14 later absorbs. Phase 15 and end-list items 4 and 6 overlap (all three are about tidying the older tabs) and could be done together.

Phase 8, next pass: drop a handle on a handle to weld

← Contents

Status	Specified in `PHASES.md`; six questions open for you
Needs first	Your answers to the questions below
Size	Small to medium
Files	`ui/interactor.js` (a hook on release), `ui/figures/figureWelds.js`, `figureLayers.js` (wiring), `figureCentroid.js` (one line), `tests/tabSwitch.mjs`

Why

The second weld bug came from the rule that a disc goes on the *active* layer only. With the Line active, a click at the Parabola's end made a free disc on the Line, and the weld that followed tied the Line to itself. You had to remember which layer was active, in points mode, before a right-click. Dropping a handle on a handle removes all of that: you are already holding the thing you want to join.

What will be built

1. **A release hook in the interactor.** `interactor.js` already has `beforeMove` and `afterMove`. It gets one more optional setting, `onRelease(index)`, called when the mouse comes up after a real drag (not a click, not an arrow key). The Draw tab passes nothing, so nothing changes there.
2. **Finding the target.** On release, Figures looks for a handle of any *visible* layer (the active one included, for a corner) whose canvas position is within the grab range of the dropped handle. Every layer's handles are mapped through its own transform first, the same way `figurePoints.js` maps arms, so a moved or turned layer works.
3. **Snapping.** A weld needs its members to coincide exactly, so the dropped handle is moved the last few pixels onto the target, through `moveHandle`, so any welds the dropped handle already has follow. Only the handle being dragged moves; the target never does.
4. **Making the weld.** If either handle has no disc, one is placed (an anchored disc at the arm end: the same record a points-mode click makes). Then the two discs are joined by the same code the right-click uses (`toggleWeldAt`'s join half, pulled out into its own function so both callers share it). If one of them is already in a group, the other joins that group; if both are in different groups, the groups merge.
5. **Telling you.** The same caption-bar message as now ("Welded"), and the weld + appears.
6. **The weld + made visible.** Today the + is drawn on the main canvas and the handle's red dot is drawn over it on the interaction layer, so all you see is a white ring. Proposed: draw the + on the interaction layer, after the handles, slightly larger than a handle (arms of 7 px). That also makes it the thing your eye lands on.
7. **The centroid's button check.** `figureCentroid.js`'s mousedown gets the same one-line `if (e.button !== 0) return;` the interactor got, so a right-press on the diamond no longer starts a move.

Questions for you (from `PHASES.md`, with what I propose):

1. **When.** On release only, so passing over a handle on the way somewhere does not weld it. (Proposed: yes.)
2. **How close.** Within the 10 px grab range, then snap. (Proposed: yes.)
3. **Discs.** Placed automatically on both handles and left visible like any other disc. (Proposed: yes.)
4. **Which handles.** Arm ends only. A Line's midpoint handle slides the whole line and is not a place a disc can go; a Parabola's midpoint handle sometimes is. (Proposed: arm ends only.)
5. **Same object.** Dropping a Parabola's one corner handle on its other corner ties the corner, exactly as a right-click does now. (Proposed: yes.)
6. **Undoing.** Right-click on the weld still removes it. A drop that welds by accident has no other undo until the undo work (see Phase 10). Acceptable for now?

One more I'd add: **should it work in points mode too?** Handles are off in points mode today, so proposed: lines mode only, which is where you drag handles anyway.

Testing

- Two Lines: drag one end onto the other's end; a weld exists, both have discs, the ends coincide exactly.
- Drag across a handle without releasing on it: no weld.
- Release 15 px away: no weld, nothing moves.
- Parabola corner to itself: one group with both corner handles.
- Drop onto a handle already in a group: one group, three members.
- A moved and turned layer: the target is found where it is drawn.
- Draw tab: dragging still works and nothing welds.
- The + is visible: a pixel check at the weld that sees black, not only white.
- Right-press on the diamond: the layer's transform is unchanged.

Phase 8b: magic points

← Contents

Status	Not started; needs design with you
Needs first	Phase 8 settled
Size	Medium to large, depending on how many kinds
Files	`ui/figures/figurePoints.js` (candidates), the registry objects that gain points, possibly `draw/drawParametrics.js`

What it is

Today a disc can only go on an *infrastructure point*: a step along an arm that the object reports through `infrastructure()`. Plenty of the places you would want to put a nail, or weld two objects, are not steps: the middle of a Parabola's curve, the top of a loop in a cycloid, the cusp of a parametric curve. "Magic points" are those extra places, offered as candidates the same way arm steps are now.

What will be built (proposed)

1. **A second list beside the arms.** `infrastructure()` may return, besides `arms`, a list of named `points`: `{ name, x, y }` in object space, recomputed each time like everything else. A disc on one is stored as `{ kind: "magic", layer, name }`, so it follows the object the way an anchored disc does: by asking the object again, not by remembering a position.
2. **The picker considers both.** `findCandidate()` looks at arm steps and magic points together, nearest wins. Nothing else about clicking changes.
3. **Welds work unchanged** for discs on magic points, except for how a follower moves: a magic point is not a handle, so a weld driven onto one translates the whole layer (rule 3 of the weld propagation), the same as a node of a Regular Polygon today.
4. **Kinds, one at a time.** Proposed first: midpoints of arms (cheap, all straight objects), then the Parabola's vertex (the point of the envelope nearest the corner). Tops of loops and cusps belong to the parametric and cycloid objects and need their own math; each is its own small step.

Related, and a decision for you

Crossings. Your stated goal is to print points *including intersections*. The planning notes call these "first-class intersections" (items I2, I3). They are the same mechanism, one level harder: the place depends on two lines, possibly from two layers. `draw/drawParametrics.js` already has a segment-crossing test, used when thinning curves; the crossings it finds are thrown away. Proposed: decide in 8b whether crossings are in scope, and if so do crossings within one layer first.

Parametrics have no handles and no node array yet (noted since 6c). Magic points are the first route by which a parametric object gets anything to click on.

> **Questions for you:**
>
> 1. Which kinds matter first for the figures you want to make?
> 2. Are crossings in 8b, later, or never?
> 3. Should magic points show as faint marks in points mode, so you know they are there, or only when the cursor is near?

Testing

For each kind: its computed position against hand-worked numbers for a default object; a real click lands a disc there; the disc follows when the object is reshaped, moved and turned; a weld to it moves the partner.

Phase 9: print aids

← Contents

Status	Not started; needs your answers on nail size and how warnings show
Needs first	Best after Phase 10 (print a saved figure)
Size	Medium
Files	new `ui/figures/figurePrint.js`; `draw/drawUtilities.js` (final mode); `figuresCommands.js` (menu items)

What it is for

The end goal is a template you lay on wood and pound nails through. Final mode already draws only the points. What is missing is everything that makes the paper usable: knowing it is the right size, knowing no two nails are too close to hammer, and getting a large figure onto letter-size sheets.

What will be built

1. **A scale.** The figure is in canvas pixels; paper is in inches or millimetres. A figure gets one number, *pixels per inch* (proposed name `figure.scale`), set from a dialog: "this figure should be N inches wide", or "this distance is N mm". It is saved with the figure (Phase 10). Nothing in the drawing changes; only printing uses it.
2. **A scale bar.** In final mode, a bar of known length (1 inch or 2 cm, with tick marks) is drawn in a corner, so after printing you can lay a ruler on it and see whether the printer scaled the page. This is the single most useful check, because browsers and printers love to "fit to page".
3. **Minimum nail-spacing check.** Every printed point is collected (discs and, in final mode, stitch endpoints; a weld group counts once, since it is one nail). Any two closer than the minimum spacing are flagged. The minimum comes from the nail: head diameter plus room for the hammer, a setting you give.
4. **Printing at size.** You said you print just the points, at size, usually in quarters, and join the sheets. Proposed: a "Print at size" command that renders the points at the figure's scale onto as many letter pages as needed, each with a small overlap, crop marks, and a page label ("2 of 4, top right"), as a PDF you open and print at 100%. This replaces the Snagit step rather than adding to it.

> **Questions for you (recorded as open):**
>
> 1. **Nail size.** What nails, and what minimum spacing feels right? (A default you can change.)
> 2. **Highlight or list.** Should crowded pairs be circled in red on the canvas, listed by number, or both?
> 3. Do you want the tiled PDF, or keep printing from Snagit and just get the scale bar?
> 4. Should points print as dots, crosshairs (easier to centre a nail on), or small circles?

Testing

Two discs placed a known distance apart at a known scale are flagged or not as expected; a weld group counts once; the scale bar's length in the PDF, measured in PDF units, equals one inch; a figure twice a page wide produces the right number of tiles with the overlap.

Phase 10: save and reopen a figure

← Contents

Status	Not started. A Node handler from the old wizard exists and would be reshaped
Needs first	Nothing. Undo (below) is proposed alongside
Size	Large (the biggest remaining phase)
Files	`node/createFigureService.mjs`, `node/sketchpadService.mjs`, `ui/nodeLayer.js`, `ui/figures/figureLayers.js`, `figuresNav.js`, `figuresCommands.js`, `figuresDisplay.js`, new `figureUndo.js`

What it is

Today a figure lives only in memory. Refresh the browser and it is gone. Phase 10 makes a figure a file: you name it, put it in a category, and it appears in the Figures tab's category list, where clicking it opens it again exactly as it was.

What is saved

The figure is already plain data, kept that way on purpose for this phase: `{ name, category, nextPoint, points, layers, drawOrder, activeLayer, welds }`, where each layer holds its registry id, its parameters, color, visibility and transform. That object, written as JSON, is the file. No code is generated (the old wizard generated a JS file; that idea is dropped, since data is easier to edit and never goes stale).

What will be built

1. **Save As** in the caption menu: a small dialog asking for the name and a category (existing from a list, or new). The subtab is renamed from "Untitled N" to the name.
2. **Save** for a figure that already has a file: writes it again, no dialog.
3. **The Node side.** One request, `saveFigure`, with the figure as its payload. The handler, reshaped from `createFigureService.mjs`:
   - makes the category folder if it is new, and its help mirror and manifest;
   - writes the figure to a temporary file and renames it into place, so a crash never leaves half a file;
   - adds or updates the manifest entry (manifest written the same temp-then-rename way);
   - refuses an empty or unusable name, and adds a number to a duplicate name (as you specified for the wizard: "if the name exists, a number is attached").
4. **Reopen.** Clicking a saved figure in the category list loads the JSON and rebuilds it. The rebuild is the real work: each layer needs its live drawing object (an instance) made again from its id and parameters, with the callbacks the interactor and welds need. `insertLayer()` already does this for one new layer, mixed with "add a layer". Proposed: pull out `makeInstance()` and a `rebuildInstances()` that reopen (and undo) call.

5. **Notes and Show Script for a figure.** A figure gets a `notes` text field (the "figure notes field" planned since the triage), shown by Notes; Show Script shows the JSON.

Undo, proposed alongside

Your idea: snapshots of part of `uiState`. The figure is one plain object, so before each action a copy goes on a stack (`structuredClone`); undo puts the copy back and calls the same `rebuildInstances()` that reopening uses. That shared rebuild is why I propose building them together. The full proposal, with when snapshots are taken and four questions, is the page *Undo in Figures: a Proposal* in the Help Overview PDF.

> **Questions for you (recorded as open, plus mine):**
>
> 1. Is a layer's `visible` saved, or does a reopened figure show every layer?
> 2. Can a figure be saved into a category that also holds Draw objects of the same name? (The wizard spec said yes, with no relationship between the two.)
> 3. Undo with Phase 10, or on its own sooner?
> 4. Does closing an unsaved figure ask first?

Testing

Save, reload the page, reopen: the figure is identical, compared pixel for pixel and field by field, including welds and transforms, and a weld still moves its partner after reopening. A duplicate name gets a number. A new category appears in the list. The test cleans up the files it writes (the existing tests already restore manifests with `git checkout`).

Phase 11: figure follow-ups

← Contents

Status	A placeholder in `PHASES.md`. Below is everything the notes push here
Needs first	Phase 10 for some items (Copy, Archive need files)
Size	Each item small to medium; do them as separate passes
Files	mostly `ui/figures/`; registry objects for Break apart and arcs

Nothing in the record says what "follow-ups" means as a whole. These are the items that earlier phases explicitly pushed to "Phase 11" or "later". Proposed: you pick which ones belong and their order.

---

1. Delete a layer

Layers are never deleted today; Break apart hides the original. A delete needs rules first: layer numbers are never reused; its discs go with it; any weld that loses a member drops it (and dissolves at one member, as now). Proposed: Delete Layer in the caption menu, with no confirmation if undo exists, a confirmation if not. This is also what the template X would call, if you decide it destroys rather than hides.

---

2. The grayed menu items

Show Script, Copy, Archive and Reset are in the caption menu, grayed, so the menu has its final shape. After Phase 10 they can work: **Copy** opens a duplicate as a new subtab; **Archive** moves the file to the category's archive the way the other tabs do; **Reset** is undecided (back to empty, or back to the last save?).

---

3. Break apart for more objects

Only Regular Polygon has `decompose()`. Inverse Star, Inner Star, Mystic Rose and the others would each get one the same way: their parabolas or lines handed over as separate Parabola or Line layers, checked pixel for pixel against the original as 7c was.

---

4. The curved objects

Circular Parabola and its Multi version still have no `infrastructure()`, because their outer arm is a curve. Your StringThing-era `arcPoints(a, b, r_frac, n)` suggests a new **Arc** object: two draggable ends and one curvature control, like a Parabola arm but curved. That gives discs and welds something to sit on. It only matches a circular perimeter, not an ellipse.

---

5. Smaller open items from earlier phases

- Dragging an anchored disc off its arm: should it become a free disc? (Open since 6b.)
- Scale in the layer transform, using the bounding-box corners left free in 7b.
- Welded objects with different step counts (raised in the triage).
- Boxes and Pursuit, skipped in 7d as temporary.

Phase 12: thumbnails in the Insert list

← Contents

Status	Not started
Needs first	Nothing
Size	Small to medium
Files	`ui/figures/figuresNav.js` (Insert list), a thumbnail folder, possibly the Node service

What it is

The Insert Object list is a column of names. With a dozen-plus registry objects, a name is a poor reminder of what you get. Each entry becomes a small picture of the object, and resting the mouse on it shows a larger one.

What will be built (proposed)

1. **Making the pictures.** Each registry object is drawn at its default settings onto an off-screen canvas and saved as a small PNG. The Patterns tab already does this for patterns (`writePatternThumbnail`, `missingImages.js`), so the same Node request is reused, not a new one.
2. **When.** A "Make thumbnails" command, plus making any that are missing when the list is built. Pictures are files, so the list stays fast.
3. **The list.** A grid of icons with the name underneath; on hover, a larger picture in a floating panel. Clicking inserts, as now.

> **For you:** make pictures live each time (no files, always current, slightly slower), or files (fast, need remaking when an object's defaults change)? Proposed: files.

Phase 13: sweep tool

← Contents

Status	Listed in your numbered list with no description
Needs first	A description from you
Size	Unknown

I found no description of the sweep tool in `PHASES.md`, the planning notes, or the conversations I can search. Rather than invent one, here are the two readings that fit what you have said before. Tell me which (or neither).

Reading A: sweep a parameter (my guess)

In September you described exploring a design by cycling between variants and *sweeping* one parameter, animated, to find "how much is too much", then combining. That is Sketchpad's second purpose exactly. A sweep tool would pick one slider of the active layer, a start, an end and a speed, and animate the object through that range, with a button to stop and keep the frame you like. It works on Draw objects as well as figure layers, since it only moves a slider.

Reading B: sweep to select

Drag a rectangle over the canvas to select every disc or handle inside it, then move, delete or weld them together. Useful once figures have many discs.

Phase 14: filesystem boundary refactor (C3)

← Contents

Status	Not started
Needs first	Best after Phase 10, so it absorbs the new save call
Size	Medium
Files	`ui/fileLayer.js`, `ui/nodeLayer.js`, `ui/manifest.js`, the tabs that build paths, `node/sketchpadService.mjs`

What it is

Item C3 in the triage, from an eight-step plan in your old notes. Today the browser reads files through `fileLayer.js` and writes through the Node service, and you held that line deliberately (the strict submenu structure exists to centralize Node access). You named two exceptions: the help system and the manifest subsystem. Path building is half-moved: `fileLayer.path` exists, but the tabs still build script and help paths inline.

What will be done

1. **Map every place** the app reaches a file or Node: reads, writes, manifests, help, thumbnails, saves. A table, delivered first, for you to read.
2. **One boundary.** The app calls named operations (`loadManifest`, `saveFigure`, `writeThumbnail`, `listCategory`) and never builds a path or a request shape itself.
3. **Behavior unchanged.** This is a refactor, not a redesign: same files, same formats, same screens.
4. **Tests at the boundary:** a missing file, a duplicate name, a broken manifest, a failed write, a relative path refused.
5. **Clear out the dead code** found while documenting: the fileLayer normalizers, the unreachable `drawRegistry` branch in `manifest.js`.

Phase 15: older tabs on the figuresCommands pattern

← Contents

Status	Added 27 October 2026 at your request
Needs first	Nothing; best near the end, with end-list items 4 and 6
Size	Investigation small; any rebuild medium, one tab at a time
Files	`ui/figures/figuresCommands.js` as the model; each tab's commands and caption files

What you asked

You read `figuresCommands.js` and found it cleaner than the other tabs' equivalents. The phase is to investigate redoing the existing tabs using it as the pattern. As you said, nothing has been looked at yet.

What will be done

1. **Say what the pattern is.** Write down what makes `figuresCommands.js` read cleanly (its header states what it owns and what it does not; one function builds the caption menu as a list; menu items are enabled or grayed by plain conditions), so the comparison is against something specific, not a feeling.
2. **Compare each tab** (Home, Draw, Patterns, Gallery, Utilities) against it: where its command and caption code lives, how its menu is built, how much each would change.
3. **Report** with a recommendation per tab: redo, partly redo, or leave alone. Some of the older code may be different for a reason the pattern does not cover.
4. **Rebuild only what you choose**, one tab per branch, with the tab tests run each time and behavior unchanged.

This overlaps with end-list items 4 (simplify tab activation) and 6 (one save and restore shape), which tidy the same files. Proposed: do the three together.

The end-of-work list

← Contents

From `PHASES.md`, in your order. These are smaller and mostly independent.

1. **Check Coloris**, the downloaded colour picker: version, that it loads, that it works without the dev server.
2. **Update `validateDrawRegistry`** once the JSON shape is stable, and add the rule that registry draw code draws only through `drawLine`.
3. **Cycloid radius control** does not change the drawn size. Likely cause: `update()` does not set what `createNodes()` reads. A candidate fix exists on an old branch; it must be tested through the real slider.
4. **Simplify tab activation:** Home starts with `saved` empty like the others, so the special START case in `setUI.js` goes; remove dead blocks. Its own small branch, since a mistake breaks the first screen.
5. **Utilities shows its last result again** when you come back to the tab.
6. **One restore shape for all tabs:** no tab's `save()` is ever called today; delete the dead save and restore code and document the one convention that is really used.
7. **Figure templates:** ready-made guide shapes (rectangle, circle) as template layers, left out of Show All and the print; a temporary ruler between any two places; a "Create bounding box" template; a pulldown on the caption bar to insert one; an X to dismiss (hide, proposed). Design with you first.
8. **A Utilities command for the global hit radius**, optional; dropped if unused.
9. **HTML help pages**, always last. Ask which pages first. (Much of this was done today in the Help Overview update; the remaining question is which further pages you want.)